

Aavaran: setup and demonstration

Two audiences in one document. Part 1 is how to show this to somebody in ten minutes. Part 2 is how the team installs and tests it properly.

Every step below has been run. Where something has not been verified, it says so.

Part 1: showing it to Ma'am

Pick the right level

	What she does	Needs	Time	Shows
A. You drive	Watches your laptop	Nothing	5 min	Everything, including the agent acting
B. She installs	Loads one folder into Chrome	Chrome only	1 min	The privacy claim, on any site she picks
C. Full agent	Installs a 6 GB model	Chrome, Node, Ollama	30 min	Everything, on her machine

Do A and B. Skip C unless she asks. B is the one that convinces, because the website is hers, not ours.

Option B, step by step

Send her `privacy-agent-extension.zip` (**1.63 MB**, so it attaches to an email). Build it with `./scripts/package-extension.sh`. It contains a `READ-ME-FIRST.txt` saying the same things, so she does not need this document.

1. Unzip it.
2. Open Chrome, go to `chrome://extensions`
3. Turn on **Developer mode**, top right
4. Press **Load unpacked**
5. Choose the unzipped folder, the one holding `manifest.json`

That is the whole installation. No account, no key, no model, no terminal, no internet.

Expect the security question, and answer it first

She spent nearly four years as a data scientist at a cybersecurity firm. An unsigned extension, loaded in developer mode, asking for permission to read every site she visits, is exactly what her professional instinct says never to install.

Raise it before she does. The zip's `READ-ME-FIRST.txt` now opens with the full permission list, why each one is needed, and what the extension does not do: no network request during a scan, no storage of any value it finds, no telemetry, no account.

The line worth saying out loud: *"open DevTools, Network tab, then press Scan. Nothing appears. That is the demonstration."* Inviting her to check beats any assurance.

If she would still rather not install it, that is a reasonable position. Show her `demo/why.html`, which needs nothing at all.

Running the demonstration

6. Open **any website with a form she can see**. A login page is ideal.
7. Type something into a field: a name, a phone number, a PAN. Use a made-up value such as ABCPE1234F, never her own.
8. Click the extension icon in the toolbar. A panel opens on the right.
9. Press **Scan this page**.

The panel lists every personal value found, what would have been sent instead, and how many bytes would have left the machine. **Nothing is transmitted**. The scan stops at the point just before a request would be made.

The one thing that can go wrong

Type into a field she can SEE. Hidden inputs are deliberately ignored, so nothing appears for them. That is correct behaviour and it looks like a failure.

Real example, measured: on `india.gov.in` the only inputs are three invisible mobile search bars, so a scan there finds nothing. The tool says so rather than pretending: *"no visible form field to type into, so there was nothing to redact."*

A site that works, and is the actual use case

The Income Tax e-filing login, `eportal.incometax.gov.in`. Its login box is literally named `panAdhaarUserId`. Type a PAN into it and scan. Verified on the live site:

```
OK: scan of https://eportal.incometax.gov.in/iec/foservices/#/login
    101 elements read, 13079 bytes would be sent
        pruned (hidden) Vikram Sharma
        redacted       ABCPE1234F
```

Use a **fake** PAN, not hers. ABCPE1234F has the right shape and belongs to nobody.

What to say while she looks at it

Keep it to three sentences, then let the panel do the work.

"This is any website, not one we prepared. I have typed a PAN into the login box. The panel shows what would be sent to the AI if we asked it to help: the PAN has become a label saying a valid PAN is present, and the value itself never leaves the machine."

If she asks how she knows nothing was sent: the scan never makes a request at all, and the repository contains an automatic check that searches the entire outgoing message for every value that was hidden.

If she asks whether it works on harder pages: yes, and the report gives a number for it, 75% recall on pages we did not write, along with why.

If you want to show the agent acting, not just redacting

That is option A on your own laptop, with the model already running and warm. Open the panel, type a goal, press **Run on this tab**. Expect about half a minute for a three-field form, and say so beforehand: it is slower than doing it by hand, and the report says that too.

It completes most runs but not all — measured between two in three and seven in eight depending on the page. If it stalls, that is the miss. Have the recorded replay open in another tab as a fallback: `demo/replay.html`, a real recorded run you can step through, which needs nothing at all.

Part 2: the team setting up

What you need

- **Node 22 or later** and **Python 3**, for the build and the tests
- **Chrome**, for the browser checks
- **Ollama** plus a 6 GB model, only for running the agent. Not needed to build or test

`./setup.sh` checks all of these and tells you what is missing rather than failing obscurely.

First run

```
git clone https://github.com/AavaranAI/Aavaran.git
cd Aavaran
./setup.sh
./test-all.sh
```

`setup.sh` installs dependencies and builds the extension. `test-all.sh` runs **19 checks** in about a minute and needs no browser and no network.

If anything fails on your machine, that is a real finding worth reporting. Two bugs have already been caught that way, both of which existed only because one developer's machine happened to have something installed.

The full run

```
ollama serve &
ollama pull qwen2.5vl:7b
(cd server && .venv/bin/uvicorn main:app --port 8975 &)
./test-all.sh --full
```

26 checks, including real browsers and the live model. Slower, and worth doing before any demonstration.

Loading the extension yourself

```
node build.mjs
```

Then `chrome://extensions`, Developer mode, Load unpacked, choose the `extension/` folder. Note that `extension/dist/` is not in git, so the build step is required. This is exactly why Ma'am gets a zip instead.

Scanning a live site from the command line

```
SIH_LIVE_URL="https://eportal.incometax.gov.in/iec/foservices/#/login" \
bash spikes/i-live-site/run.sh
```

It opens the page in a real browser, types realistic values into whatever inputs it finds, runs the production scan, and reports what happened to each value: redacted, pruned because the box was

invisible, never extracted, or leaked. Only a leak fails the run, and every outcome is printed so none of them can hide.

Building the things you hand over

```
./scripts/package-extension.sh      # the installable zip for Ma'am
node demo/build-demo.mjs           # the recorded replay, demo/replay.html
node bench/make-wild-holdout.mjs    # regenerate the wild test corpus
```

Troubleshooting

Symptom	Cause	Fix
Scan finds nothing	The field is hidden	Type into one you can see
"cannot scan this page"	A chrome : // or extension page	Open a normal website
Extension will not load	dist/ missing	Run node build.mjs, or use the zip
"Run on this tab" errors	Model server not running	Start Ollama and uvicorn, or just use Scan
Agent stalls mid-task	The one run in eight	Say so, and fall back to demo/replay.html
A test fails only on your machine	Genuinely interesting	Report it, do not work around it

What is honest to claim

- The extension runs on **Chrome and Firefox**, on Windows, macOS and Linux.
- The **scan needs no model and no network**, and works on any site.
- On pages we did not write, **PII recall is 100%** across 39 labelled values and **redaction precision is 97.5%**, with one known false positive.
- The agent completes a real multi-step task in a **median 31.5 seconds**, in **seven runs out of eight**.
- Every number here is reproducible from the repository with one command.

Do not claim it is unbreakable, that it defends against a compromised machine, or that all eleven languages have been verified. Eight have not, and it is in the report.